Verification, Canonicalization, and Equivalence
in Quaternionic Quantum Compilation

John G. Van Geem
RQM Technologies
orcid: 0009-0002-4003-8452

Date 2026-04-09

Abstract

This paper addresses the trust problem for quaternionic quantum compilation. The preceding papers in the series established a fixed quaternion–SU(2) convention, a canonical single-qubit intermediate representation $\mathrm{u1q}(w, x, y, z)$, and a local fusion pass for single-qubit circuit segments. The present paper defines how optimized outputs are verified. The core proposal is an authoritative canonical equivalence criterion: two single-qubit segments are treated as equivalent when their canonical quaternionic representatives agree after normalization and sign-canonicalization on the quotient $q \sim -q$ induced by global-phase removal. Around that criterion, the paper defines preserved invariants, validation procedures, and interoperability diagnostics with matrix-based toolchains. In particular, it introduces a phase-aware matrix residual, a trace-overlap diagnostic, and a process-fidelity-style check under the fixed quaternion–SU(2) map. A small shipped verification harness evaluates both positive and negative cases. On 7 measured comparison cases, including exact named-gate recovery, exact identity elimination, generic-u1qverification, wrong-axis substitutions, and perturbed generic outputs, all 4 positive cases pass the authoritative canonical test and all 3 negative cases are rejected. The paper argues that quaternionic compilation can therefore be audited by deterministic canonical output forms while remaining interoperable with standard matrix-based verification workflows.

Keywords: quantum compilation verification; canonical equivalence; single-qubit segments; SU(2); unit quaternions; process fidelity diagnostics

1 Introduction

Optimization invites a trust question. Once a compiler replaces one gate list by another, how does one know that the transformation is correct? For quaternionic compilation the question is especially predictable, because the optimization language is not the familiar matrix syntax of most toolchains. Papers 2 and 3 in this series showed that single-qubit segments admit a deterministic quaternionic internal form and that adjacent one-qubit runs can be fused by quaternion multiplication. The present paper asks the verification question that follows immediately from those results.

The answer proposed here has two layers. The first layer is *authoritative* inside the quaternionic compiler: compare canonical representatives. If two optimized segments canonicalize to the same quaternionic payload, then they are equivalent modulo global phase under the fixed convention of the series. The second layer is *interoperable*: expose matrix-based diagnostics that can be checked by standard toolchains, such as phase-aware residuals, trace overlap, and process fidelity under the same SU(2) representative.

The point is not to replace matrix-based verification. The point is to make quaternionic compilation auditable in its own native representation while remaining transparent to partner systems, SDKs, and review workflows that still speak in matrices. This paper therefore treats verification as part of the compiler method itself.

Its scope is deliberately narrow. It concerns single-qubit segments only. It does not define verification of entangling-gate internals, does not claim a universal proof framework for all quantum optimizations, and does not claim that one diagnostic alone is sufficient for every integration setting.

2 Background dependency on Quaternionic Quantum Computing

First we fix the quaternion–SU(2) convention:

$$\Phi(a + b\mathbf{i} + c\mathbf{j} + d\mathbf{k}) = \begin{pmatrix} a - di & -c - bi \\ c - bi & a + di \end{pmatrix} = aI - i(b\sigma_x + c\sigma_y + d\sigma_z). \tag{1}$$

Under this map, the unit quaternions model SU(2) exactly, and every single-qubit gate action modulo global phase is represented by a sign class $\{q, -q\}$ of unit quaternions.

Second we define the canonical IR object

$$\mathrm{u1q}(w, x, y, z)$$

with the canonicalization rule

$$\mathrm{Can} = \mathrm{SignCan} \circ \mathrm{Norm},$$

where Norm restores unit norm and SignCan chooses the preferred representative by the rule $w > 0$, with an explicit deterministic tie-break on the equator $w = 0$.

Third we define the local fusion operator for single-qubit segments

$$(\mathrm{seg}) = \mathrm{Can}(q_n \cdots q_1)$$

and used canonical quaternion equality as the authoritative equivalence criterion for fused outputs. The present paper turns that criterion into a formal verification layer.

3 Problem setup: the trust layer

Definition 3.1 (Single-qubit segment)**.** *A* single-qubit segment *on wire a is a maximal consecutive run of gates that act only on wire a, bounded by the beginning or end of the circuit, or by a boundary operation outside the present single-qubit model for that wire, including entangling gates, measurement, reset, barrier, or classical-control boundaries.*

Definition 3.2 (Verification problem)**.** *Given an original single-qubit segment* seg *and a candidate optimized segment* seg$'$*, the verification problem is to determine whether* seg *and* seg$'$ *represent the same single-qubit action modulo global phase under the fixed quaternion–*SU(2) *convention.*

Remark 3.3. This paper treats verification as a relation between *segment actions*, not between textual syntaxes. Two segments may use different named-gate spellings, different reconstruction policies, or a generic u1q primitive, yet still be equivalent at the represented-action level.

4 Canonical equivalence and preserved invariants

4.1 Authoritative canonical criterion

Definition 4.1 (Canonical segment representative)**.** *Let* $\mathrm{seg} = (g_1, \ldots, g_n)$ *be a single-qubit segment in application order, and let* $q_1, \ldots, q_n$ *be the translated unit-quaternion representatives of those gates. The canonical segment representative is*

$$Q(\mathrm{seg}) = \mathrm{Can}(q_n \cdots q_1).$$

Definition 4.2 (Authoritative equivalence)**.** *Two single-qubit segments* seg *and* seg′ *are* canonically equivalent*, written*

$$\mathrm{Verify}(\mathrm{seg}, \mathrm{seg}'),$$

if

$$Q(\mathrm{seg}) = Q(\mathrm{seg}').$$

Proposition 4.3 (Canonical equivalence is exact segment equivalence modulo global phase)**.** *For single-qubit segments,* $\mathrm{Verify}(\mathrm{seg}, \mathrm{seg}')$ *holds if and only if* seg *and* seg′ *represent the same* SU(2) *action up to the global-phase quotient inherited from* U(2)*.*

Proof. By paper 1, single-qubit actions modulo global phase correspond exactly to sign classes $\{q, -q\}$ of unit quaternions under the fixed map (1). By paper 2, Can chooses exactly one representative from each sign class. Hence two segments produce the same canonical representative if and only if they produce the same sign class, which is exactly the same single-qubit action modulo global phase. □

4.2 Preserved invariants

Definition 4.4 (Quaternionic compilation invariants)**.** *For a valid optimized replacement of a single-qubit segment, the following invariants are preserved:*

(a) unit norm after normalization;

(b) canonical sign choice under the fixed hemisphere rule;

(c) represented SU(2) *action modulo global phase; and*

(d) reproducible canonical output form.

Remark 4.5. The final item is operationally important. Even when multiple frontend or presentation-level reconstructions are available, the canonical quaternionic payload is unique. This is what makes optimized outputs reproducible across runs and comparable across implementations.

Proposition 4.6 (Canonical output form is deterministic)**.** *If two compilation runs produce valid optimized representations of the same single-qubit segment action under the fixed convention, then their canonical quaternionic payloads are identical.*

Proof. Both outputs represent the same sign class $\{q, -q\}$ in the unit quaternions. The canonicalization map of paper 2 chooses exactly one representative from that class. Therefore the payload is unique. □

5 Validation procedures

5.1 Primary validation procedure

Definition 5.1 (Canonical validation procedure)**.** *To validate a candidate optimized segment* seg' *against an original segment* seg*:*

(1) translate each segment into the fixed quaternionic representation;

(2) multiply in application order to obtain raw segment products;

(3) normalize and sign-canonicalize both products; and

(4) compare the resulting canonical representatives for equality.

Remark 5.2. This procedure is the authoritative one for the present series. It is exact in the mathematical model and becomes an implementation check after explicit tolerance rules are layered on top.

5.2 Tolerance-aware numerical variant

Definition 5.3 (Tolerance-aware canonical check)**.** *In floating-point settings, treat two canonical representatives as equal when their coordinatewise difference is below a declared tolerance after normalization and sign-canonicalization.*

Remark 5.4. The tolerance belongs to the implementation policy, not to the exact equivalence relation. The exact mathematical criterion remains equality of canonical representatives.

6 Interoperability with matrix-based toolchains

Trust often requires compatibility with existing verification culture. For this reason the paper introduces diagnostics that can be computed from the same segment pair after applying the fixed map (1).

Definition 6.1 (Phase-aware matrix equivalence)**.** *Let* $U, V \in \mathrm{SU}(2)$ *be the matrix representatives of two single-qubit segments. Define*

$$\mathrm{MatrixEq}(U, V)$$

to hold when $U^\dagger V$ *is equal to a scalar phase times the identity.*

Definition 6.2 (Trace overlap and process fidelity)**.** *For* $U, V \in \mathrm{SU}(2)$ *define the trace-overlap diagnostic*

$$\mathrm{TraceDiag}(U, V) = \left|\mathrm{Tr}(U^\dagger V)\right|$$

and the process-fidelity-style quantity

$$\mathrm{ProcFid}(U, V) = \frac{\left|\mathrm{Tr}(U^\dagger V)\right|^2}{4}.$$

Definition 6.3 (Phase-aware residual)**.** *Let* $\lambda = \mathrm{Tr}(U^\dagger V)/|\mathrm{Tr}(U^\dagger V)|$ *when the trace is nonzero. Define the phase-aware residual*

$$\mathrm{PhaseRes}(U, V)$$

as the maximum-entry norm of $U^\dagger V - \lambda I$*.*

Proposition 6.4 (Canonical equality implies perfect matrix diagnostics)**.** *If* $\mathrm{Verify}(\mathrm{seg}, \mathrm{seg}')$ *holds exactly, then for the corresponding matrix representatives* U *and* V *one has*

$$\mathrm{MatrixEq}(U, V), \qquad \mathrm{ProcFid}(U, V) = 1, \qquad \mathrm{TraceDiag}(U, V) = 2, \qquad \mathrm{PhaseRes}(U, V) = 0.$$

Proof. Canonical equality implies the same unit quaternion representative. Under (1), the two segments therefore produce the same SU(2) matrix, so $U^\dagger V = I$. The stated diagnostics follow immediately. □

Remark 6.5. The converse must be handled with care numerically. Near-equality in trace or fidelity can diagnose closeness, but the authoritative exact yes/no criterion in this paper remains canonical quaternion equality after canonicalization.

7 Worked examples

Example 7.1 (Exact named-gate recovery). Compare

$$R_x(\pi/2); R_x(\pi/2) \qquad \text{and} \qquad X.$$

Both sides canonicalize to the quaternion $(0, 1, 0, 0)$. Hence $\mathrm{Verify}(\mathrm{seg}, \mathrm{seg}')$ holds exactly.

Example 7.2 (Identity elimination). Compare

$$T; T^\dagger \qquad \text{and} \qquad \emptyset.$$

Both sides canonicalize to $(1, 0, 0, 0)$, so the empty output is verified as a correct identity elimination.

Example 7.3 (Generic primitive verification). Let the original segment be

$$H; T; R_z(\pi/3),$$

and let the optimized candidate be the generic primitive

$$\mathrm{u1q}(0.560985526797, -0.430459334577, -0.560985526797, -0.430459334577).$$

The two canonical quaternionic payloads agree, so the generic primitive is verified exactly even though it is not a named gate.

Example 7.4 (Negative wrong-axis case). Compare

$$R_z(\pi/3) \qquad \text{and} \qquad R_x(\pi/3).$$

The canonical representatives differ, so the authoritative canonical check fails. The matrix diagnostics also reflect non-equivalence:

$$\mathrm{ProcFid} = 0.5625, \qquad \mathrm{PhaseRes} = 0.5.$$

8 Reference verification corpus and measured diagnostics

The paper package includes the executable verifier `artifacts/code/verify_canonical_equivalence.py`, which generates the measured results stored in `artifacts/code/verification_results.json`. The shipped reference corpus contains 7 comparison cases:

- 4 positive equivalence cases; and
- 3 negative non-equivalence cases.

The measured aggregate outcome is:

- all 4 positive cases pass the authoritative canonical check;
- all 3 negative cases are rejected by the authoritative canonical check; and
- all 7 cases are consistent with the expected verdict.

Table 1 summarizes the reference corpus.

Case type	Cases	Expected equivalent	Passed/rejected	Consistent
Positive exact cases	4	yes	4 passed	yes
Negative distinct cases	3	no	3 rejected	yes
Total	7	mixed	7 correct	yes

Table 1: Measured verification results on the shipped reference corpus.

The individual diagnostic patterns are also informative:

(1) positive exact cases produce canonical equality, process fidelity 1, and near-zero phase-aware residuals;

(2) structurally wrong substitutions such as X versus Y produce zero trace overlap and zero process fidelity; and

(3) small perturbations of a generic u1q primitive can exhibit fidelity close to 1 while still failing the authoritative canonical equality test.

Remark 8.1. The third point is operationally important. Fidelity-like diagnostics are useful for interoperability and debugging, but they are not by themselves a substitute for exact canonical equivalence when the compiler intends an exact optimization.

9 Standard mathematics vs. RQM Technologies framing

9.1 Standard ingredients

The following ingredients are standard:

(1) single-qubit gate actions modulo global phase are modeled by SU(2) representatives;

(2) unit quaternions model SU(2) under the fixed convention of paper 1;

(3) the ambiguity in that model is exactly $q \sim -q$; and

(4) matrix overlap, trace diagnostics, and fidelity-style checks are standard verification tools.

9.2 Project-specific contribution

The project-specific contribution of this paper is the trust layer:

(a) canonical quaternion equality as the authoritative exact verification rule;

(b) fixed sign conventions and reproducible canonical output forms;

(c) explicit preserved invariants for quaternionic compilation; and

(d) a paired matrix-interoperability layer for integration with existing toolchains.

10 Scope, limitations, and non-claims

This paper does *not*:

(1) define verification of entangling-gate internals;

(2) provide a universal proof system for all quantum compiler passes;

(3) claim that trace or fidelity diagnostics alone are sufficient for every integration context;

(4) prove anything about hardware execution correctness; or

(5) replace existing matrix-based verification practice.

It defines a trust layer for single-qubit quaternionic compilation: canonical equivalence, preserved invariants, validation procedures, and interoperable diagnostics.

11 Conclusion

Quaternionic compilation becomes safer to adopt once its outputs can be verified deterministically. The central proposal of this paper is therefore simple:

(1) canonical quaternion equality is the authoritative exact equivalence criterion for single-qubit segments;

(2) preserved invariants make optimized outputs reproducible; and

(3) matrix-based diagnostics provide interoperability with standard toolchains.

In that sense, this paper supplies the trust layer for the earlier representation and optimization papers. It does not change the compiler method; it makes that method auditable.

A Recommended validation pipeline

For an exact local optimization on a single-qubit segment, the recommended verification procedure is:

(1) compute canonical quaternionic representatives for original and optimized segments;

(2) compare them for authoritative exact equality;

(3) if needed for interoperability, compute the matrix representatives under (1);

(4) record phase-aware residual, trace overlap, and process fidelity for audit logs.

B Companion code and figures

The executable verifier and its measured output are included in `artifacts/code/verify_canonical_equivalence.` and `artifacts/code/verification_results.json`. Publication-oriented SVG figures are included in `artifacts/figures/`.